1. The Core Architecture: How Docker Connects Containers

When Docker starts up on your Amazon Linux server, it automatically creates a default virtual network interface on your host OS called docker0. Think of this as a software-defined network switch. Every time you run a container, Docker plugs it into this virtual switch and assigns it a private internal IP address (usually in the 172.17.0.x range).

The Core Problem with Container IPs:

If Container A needs to send data to Container B, you *could* technically connect using Container B's internal IP address. However, containers are temporary. If Container B crashes and restarts, Docker will assign it a **brand-new IP address**. Hardcoding IPs into your code will break your production system constantly.

The Solution: Embedded DNS Service Discovery

To solve this, Docker builds a custom **DNS server** directly into its network engines. When you create a custom virtual network, you can completely ignore IP addresses. Containers can talk to one another using their human-readable **Container Name** or **Compose Service Name** as a permanent hostname!

For example, your frontend web container connects to your database simply by targeting http://database-engine:3306. Docker's internal DNS automatically resolves that name to whichever internal IP the database currently holds.

2. The 3 Network Drivers You Must Explain to Your Trainer

Docker provides different network types (drivers) depending on how isolated or open you need a container to be. These are the top 3 interview-critical network drivers:

A. Bridge Driver (Default & Most Important)

- **How it works:** Creates an isolated private network on your host server. Containers on the same bridge network can talk to each other perfectly, but they are hidden from the outside world unless you explicitly expose their ports using the -p flag.
- **Production Use:** This is what you use 99% of the time for web applications and databases.

B. Host Driver

- **How it works:** Removes the network isolation between the container and your Amazon Linux server. The container does not get its own IP address; instead, it binds directly to your host server's network ports.

- **Production Use:** Used for high-performance applications (like streaming or heavy logging tools) where you cannot afford the minor routing delay of a virtual bridge.

C. None Driver

- **How it works:** Completely disconnects the container's network interface card. The container has no access to external networks, other containers, or the internet.
- **Production Use:** Used for running ultra-secure batch processing jobs, heavy mathematical calculations, or security environments where data must never leak outside.

3. Hands-On Practical: Managing Custom Networks via CLI

Let's step through how to build, inspect, and test an isolated network environment completely by hand to show your instructor how the mechanics work.

Step 1: Create a Custom Private Bridge Network

Bash

```
docker network create enterprise-secure-lan
```

Step 2: Verify and Inspect the Subnet

Let's list your server's available networks to see it listed:

Bash

```
docker network ls
```

To see the exact private IPv4 subnet and gateway range Docker allocated for this network lane, run:

Bash

```
docker network inspect enterprise-secure-lan
```

What to look for: Look inside the IPAM -> Config block. You will see a subnet allocation such as "Subnet": "172.18.0.0/16".

Step 3: Run Two Containers on the Same Network 🚀

Let's launch two separate containers and drop them onto this custom network lane. We will name one alpha-frontend and the other beta-backend:

Bash

Launch Container 1 (Alpha) on your custom network

docker run -d --name alpha-frontend --network enterprise-secure-lan nginx:alpine

Launch Container 2 (Beta) on the same network

docker run -d --name beta-backend --network enterprise-secure-lan nginx:alpine

(Notice we didn't use the -p port mapping flag. Because they share the same network bridge, they can access all of each other's internal ports automatically while remaining completely invisible to the outside public internet!)

Step 4: Test Embedded DNS Resolution (The Magic Check) 🧪

Let's execute a shell command inside alpha-frontend and force it to ping or curl beta-backend by its **name** instead of an IP address:

Bash

docker exec -it alpha-frontend ping -c 3 beta-backend

Expected Output:

Plaintext

PING beta-backend (172.18.0.3): 56 data bytes

64 bytes from 172.18.0.3: seq=0 ttl=64 time=0.086 ms

64 bytes from 172.18.0.3: seq=1 ttl=64 time=0.112 ms

--- beta-backend ping statistics ---

3 packets transmitted, 3 packets received, 0% packet loss

It resolves perfectly! The internal Docker DNS intercepted the word beta-backend, checked the container registry list on your enterprise-secure-lan bridge, and instantly routed the traffic to the correct internal container interface.